

Access Paths for Efficient Ordering with Large Language Models

Fuheng Zhao
UC Santa Barbara
fuheng_zhao@ucsb.edu

Jiayue Chen
University of Chicago
jiayuechen@uchicago.edu

Yiming Pan
UC Los Angeles
yimingpan@ucla.edu

Tahseen Rabbani
University of Chicago
trabbani@uchicago.edu

Divyakant Agrawal
UC Santa Barbara
agrawal@cs.ucsb.edu

Amr El Abbadi
UC Santa Barbara
amr@cs.ucsb.edu

Abstract

We present the LLM ORDER BY operator as a logical abstraction and study its physical implementations within a unified evaluation framework. Our experiments show that no single approach is universally optimal, with effectiveness depending on query characteristics and data. We introduce three new designs: an agreement-based batch-size policy, a majority voting mechanism for pairwise sorting, and a two-way external merge sort adapted for LLMs. With extensive experiments, our agreement-based procedure is effective at determining batch size for value-based methods, the majority-voting mechanism consistently strengthens pairwise comparisons on GPT-4o, and external merge sort achieves high accuracy–efficiency trade-offs across datasets and models. We further observe a log-linear scaling between compute cost and ordering quality, offering the first step toward principled cost models for LLM-powered data systems¹.

1 Introduction

Recent advances in large language models (LLMs) have facilitated complex data processing, such as automatically translating natural language questions into SQL queries Zhao et al. [2024b], Floratou et al. [2024], Li et al. [2023], Gao et al. [2023], Gong et al. [2025], powering autonomous problem-solving agents Zhuge et al. [2024], Shankar et al. [2024], Liu et al. [2025], He et al. [2025], Zhao et al. [2023], Sun et al. [2025], and introducing new declarative query languages with AI operators Zeng and Yan [2025], Zhao et al. [2024a], Satriani et al. [2025], Glenn et al. [2024]. Building LLM declarative interface through semantic operators has also attracted significant attention from industry, with systems such as BigQuery ML Google Cloud [2025], Snowflake AI SQL Snowflake, Inc. [2025], and Amazon Redshift with user-defined functions Amazon Web Services [2024] providing support for these capabilities.

In this work, we focus on one specific LLM operator: **LLM ORDER BY**, commonly used for sorting or ranking. Sorting is a fundamental operation in data processing and underpins a wide range of analytical and decision-making tasks. Within the context of LLMs, **ORDER BY** enables ranking

¹Our code is publicly available at OrderByLLM

items based on semantic signals, user preferences, or task-specific criteria that are difficult to capture with traditional methods. For example, in information retrieval, it is common to use LLMs to re-rank retrieved documents or passages based on the user’s question or preferences Zhu et al. [2023]. In the context of relational data processing, LLMs can be leveraged to rank selected rows based on attributes that are not explicitly stored in the relational table, such as inferred sentiment or other factual information Zhou et al. [2024], Zhao et al. [2024a].

Given the importance of data ordering and ranking, researchers have explored a wide range of classical ideas in conjunction with LLMs. Prior works primarily study four classes of methods to implement sorting on top of LLMs: (i) **pointwise sorting** derives the value independently for each given key (e.g., passage relevance, review sentiment) Drozdov et al. [2023], Sachan et al. [2022], Zhao et al. [2024a], (ii) **pairwise sorting** compares two keys and then determines the best out of two keys Qin et al. [2023], Luo et al. [2024], Shah and Wainwright [2018], and (iii) **listwise sorting** considers an entire set of keys simultaneously to produce a ranked order Ma et al. [2023], Sun et al. [2023]. There is no clear consensus on which approach is best. For example, Lotus Patel et al. [2024], a recent LLM-powered data processing system, implements LLM ORDER BY using a pairwise approach and finds that it achieves strong performance. However, other works find listwise sorting often provide superior performance Ma et al. [2023], Pradeep et al. [2023]. At the same time, many criticize the pointwise approach when applied with off-the-shelf LLMs (e.g., ChatGPT), since these models lack calibrated prediction probabilities, which often leads to unstable or unreliable rankings Qin et al. [2023].

Thus, in this paper, we aim to address the following question: **Given a logical LLM ORDER BY operator, how should it be implemented physically?** To this end, we conduct a systematic evaluation of existing approaches and our new proposed algorithms within a unified experimental framework across different tasks. Contrary to many previous findings, we observe that pointwise sorting (often criticized for its limited accuracy) can in fact achieve strong accuracy in certain scenarios, while pairwise and listwise approaches excel in others. This highlights the importance of tailoring the ranking strategy to the task setting rather than relying on a one-size-fits-all solution. This also suggests that the physical **LLM Access Path**² should be carefully selected based on the query characteristics and task requirements. While this paper does not aim to optimize access path selection for all operators, we present empirical evidence that identifies which categories of sorting tasks derive the greatest benefit from particular implementations. Moreover, after examining the efficiency–effectiveness trade-offs across different approaches, we conjecture a log-linear scaling relationship between compute cost and quality in sorting. Furthermore, we introduce several enhancements to existing algorithms and propose new algorithms. We proposed a new listwise approach based on the two-way external merge sort, a classical merge sort algorithm designed for handling data that exceeds context (main-memory) capacity Selinger et al. [1979]. We find external merge sort consistently offer superior accuracy and computation trade-offs among all algorithms.

To conclude, our contributions can be summarized as follows:

- We observe that the physical implementation of **LLM ORDER BY** depends on the user query and underlying data. **No single approach is universally optimal.**
- We propose new strategies for determining the batch size, introduce a variant of the pairwise algorithm that incorporates majority voting, and extend the classical two-way merge sort to the LLM-based Order By setting.
- Based on our empirical observations, we conjecture a log-linear scaling law between compute cost and quality of the ranking order

2 Preliminary

The **LLM ORDER BY** operator takes as input a list of keys along with a ranking criterion and produces an ordered list. The keys may represent diverse items, such as documents, reviews, or database rows, while the ranking criterion specifies the basis for ordering (e.g., relevance to a query). Unlike traditional database settings where the ordering attribute is explicitly stored in the table, LLM-based ordering relies on semantic judgments or inferred attributes.

Although LLM ranking algorithms are typically designed for passage or document ranking and are commonly categorized as pointwise, pairwise, or listwise, in this paper we adopt a slightly different

²originating from classical data base systems Selinger et al. [1979]

Algorithm 1: Determine Batch Size for External Pointwise

Input: Dataset D , LLM function LLM , agreement threshold θ , maximum size M

$m \leftarrow 2$;

while $2m < |D|$ **and** $m < M$ **do**

$B_1 \leftarrow D[:m]$ // first m keys

$B_2 \leftarrow D[m+1:2m]$ // next m keys

$B_3 \leftarrow B_1 \cup B_2$ // combined batch of size $2m$

 Run LLM on B_1, B_2, B_3 to get V_1, V_2, V_3 ;

if any output invalid **then**

break;

 Merge $V_{12} = V_1 \cup V_2$;

 Compute agreement $\alpha = \#agree(V_{12}, V_3)/(2m)$;

if $\alpha \geq \theta$ **then** $m \leftarrow 2m$;

else return m ;

return m ;

how agreement calculated?

terminology. We refer to pointwise methods as **value-based**, pairwise methods as **comparison-based**, and further divide listwise methods into two variants: **external value-based** and **external comparison-based**. The intuition is that listwise methods can be viewed as batching multiple queries: either batching the derivation of values for multiple keys (external value-based) or batching the determination of the correct permutation (external comparison-based). We also note that the external comparison-based approach is also termed **setwise** Zhuang et al. [2024].

In addition to used terminology, our work specifically focuses on ranking through the generation API rather than relying on logits. While some pointwise and listwise methods utilize logits to produce calibrated scores, such approaches are often impractical with off-the-shelf LLMs (e.g., GPT-4o), as many such models expose only generation interfaces without access to logits. Moreover, existing LLM-based data analytical systems Shankar et al. [2024], Liu et al. [2025] frequently rely on off-the-shelf models due to their accessibility, reliability, and ease of integration. Consequently, we restrict our study to methods that can be implemented using the generation API, ensures that our findings apply broadly to real-world LLM powered data systems. We leave the investigation of logit-based methods and calibration techniques to future work.

3 Methodology

In this section, we present our methodology for implementing different physical realizations of the **LLM ORDER BY** operator.

3.1 Value-Based Ordering

In value-based ordering, the LLM assigns a score or value to each key, and the final ranking is obtained by sorting the keys according to these values. This approach is commonly used in LLM-powered relational data systems Zhao et al. [2024a], Alaparthi et al. [2025], Glenn et al. [2024], where tables may contain millions of rows, as the value-based method requires the $O(N)$ LLM function calls where N is the number of rows. To even further reduce the number of LLM function calls, a commonly used technique is **external pointwise approach**, where m keys are batched into a single prompt and the model returns a list of values. With the external pointwise approach, the number of LLM function calls becomes $O(N/m)$.

However, determining an appropriate batch size remains an open challenge. We therefore propose a simple agreement-based procedure to adaptively choose the batch size for a given task. As shown in Algorithm 1, we start with a batch of two keys and repeatedly double the batch size as long as the model's output of the individual key remain consistent between two separate batches of size m and one combined batch of size $2m$. We have added a local client-side LLM output cache. Repeated prompts with identical inputs are served directly from the cache, reducing redundant LLM function calls, and the number of LLM function calls in Algorithm 1 becomes $O(\log_2 m)$. We also note that LLMs may occasionally fail to produce valid outputs (e.g., returning fewer or more values than the

Algorithm 2: Quick Sort with Majority Voting

Input: Dataset D , LLM comparator COMPARE, votes $v \geq 1$
Choose pivot $p \leftarrow D[1]$;
Initialize $L \leftarrow [], R \leftarrow []$;
for each item $x \in D[2:]$ **do**
 $r \leftarrow \text{COMPARE}(x, p)$ // initial comparison v.s. pivot
 if $r = L$ **then**
 $P \leftarrow G$
 else
 $P \leftarrow L$
 $\mathcal{Y} \leftarrow \text{SAMPLE}(P, v-1)$ // randomly sample $v-1$ keys
 $\text{votes} \leftarrow [r]$ // collect up to v votes
 for each $y \in \mathcal{Y}$ **do**
 $\text{votes} \leftarrow \text{votes} \cup \{\text{COMPARE}(x, y)\}$ // aggregate comparisons
 if $\text{MAJORITY}(\text{votes}) = L$ **then** $L \leftarrow L \cup \{x\}$;
 else $G \leftarrow G \cup \{x\}$;
 $S_L \leftarrow \text{LLMQUICKSORT}(L, v)$;
 $S_R \leftarrow \text{LLMQUICKSORT}(R, v)$;
return $S_L \parallel [p] \parallel S_R$;

number of input keys). In such cases, the algorithm terminates early. Putting it together, we first make $O(\log_2 m)$ LLM calls to determine the appropriate batch size, and then perform $O(N/m)$ LLM calls to obtain the values for all keys.

Algorithm 3: LLM External Merge Sort

Input: Dataset D , batch size m , sorter $\text{LLMSORT}(\cdot)$, two-way merger $\text{LLMMERGE}(\cdot)$
Phase 1: Run generation;
Partition D into chunks $C_1, \dots, C_m$ of size at most m ;
for $i \leftarrow 1$ **to** m **do in parallel**
 $S_i \leftarrow \text{LLMSORT}(C_i)$ // listwise sorting
 $\text{Runs} \leftarrow [S_1, \dots, S_m]$;
Phase 2: Iterative merging;
while $|\text{Runs}| > 1$ **do**
 $\text{NewRuns} \leftarrow []$;
 for $i = 1, 3, 5, \dots, |\text{Runs}|$ **do**
 if $i + 1 \leq |\text{Runs}|$ **then**
 $S \leftarrow \text{LLMMERGE}(\text{Runs}[i], \text{Runs}[i+1], m)$ // Algorithm 4
 append S to NewRuns ;
 else
 append $\text{Runs}[i]$ to NewRuns // odd run carried forward
 $\text{Runs} \leftarrow \text{NewRuns}$;
return $\text{Runs}[1]$ // final sorted sequence

3.2 Comparison-Based Ordering

In comparison-based ordering, the LLM is asked to directly compare two or more keys and decide the ranking order. Instead of assigning independent values to each key, the model outputs a permutation of the input keys. Compared to value-based ordering, comparison-based methods are typically more expensive, as the number of comparisons can grow quadratically with the number of keys. Commonly, researchers have implemented quick sort and bubble sort on top of the LLM comparators. For instance, the external bubble sorting algorithm Sun et al. [2023]³ sorts m keys at a time and then

³Also referred to as the Sliding Window Strategy

Algorithm 4: LLM Two-Way Merge

Input: Sorted runs L_1, L_2 ; batch size m ; sorter $\text{LLMSORT}(\cdot)$
 $i \leftarrow 0, j \leftarrow 0$ // indices in L_1 and L_2
 $S \leftarrow []$;
 $h \leftarrow \max(\lfloor m/2 \rfloor, 1)$;
while $i < |L_1|$ **or** $j < |L_2|$ **do**
 if $i \geq |L_1| \vee j \geq |L_2|$ **then**
 $\perp$ **append** $(i \geq |L_1| ? L_2[j:] : L_1[i:])$ **to** M ; **break**
 // Store h items from L_1 and L_2
 $B \leftarrow$ up to h items from $L_1[i:]$ and $L_2[j:]$;
 $R \leftarrow \text{LLMBUFFERSORT}(B)$
 // Emit in order until one side is exhausted
 $e_1 \leftarrow 0, e_2 \leftarrow 0$;
 for each $x \in R$ **do**
 $\perp$ **append** x **to** S ;
 if $x \in L_1$ **then** $e_1 \leftarrow e_1 + 1$;
 else $e_2 \leftarrow e_2 + 1$;
 if $e_1 = h$ **or** $e_2 = h$ **then**
 $\perp$ **break**
 $i \leftarrow i + e_1, j \leftarrow j + e_2$;
return M

advances by $\frac{m}{2}$ steps. It allows the model to repeatedly compare overlapping groups of keys (i.e., $\frac{m}{2}$ keys), so that items of higher rank gradually 'bubble up' in successive windows.

In this subsection, we introduce two new comparison-based algorithms: (i) QuickSort with majority voting and (ii) an LLM-based external merge sort. The first is detailed in Algorithm 2. As shown in Algorithm 2, our LLM Quick Sort algorithm follows the classical divide-and-conquer strategy and use a LLM comparator to determine the sorted permutation of the given keys. To further improve robustness, we validate each comparison through majority voting, where the item is compared not only with the pivot but also with additional sampled peers, thereby reducing the risk of misclassification from a single erroneous LLM output. The high-level intuition is that if a key is less than the pivot, it should also be less than any item greater than the pivot, and vice versa.

Moreover, we observe that the external bubble sort Sun et al. [2023] remains quadratic. To address this limitation, we propose using the classical external merge sort approach. External merge sort employs a two-stage process: first, the input is divided into sorted runs, and second, these runs are merged iteratively to produce the final sorted order. As shown in Algorithm 3, our external merge sort procedure organizes the overall computation by first generating sorted runs using LLM with batch size m and then repeatedly merging them until a single fully sorted sequence is obtained. The merging itself is carried out by the two-way merging procedure described in Algorithm 4, which leverages a sliding window to rank a small set of keys at a time using the LLM. By combining these two components, the algorithm achieves scalability beyond quadratic approaches and keep the number of LLM calls manageable.

4 Experiments

In this section, we evaluate our proposed algorithms alongside existing baselines on two different datasets. We report results in terms of both effectiveness (ranking quality) and efficiency (number of LLM calls and tokens), highlighting the trade-offs among value-based and comparison-based approaches (i.e., pointwise, pairwise, and listwise).

4.1 Datasets

The two dataset we used are:

- NBA: It contains 200 NBA players name and height information OpenIntro [2025]. We also manually validated and corrected erroneous height entries using official player information available on Wikipedia. The goal is to order all players by their heights.
- DL19: TREC-DL19 is a widely used benchmark dataset in information retrieval literatures Craswell et al. [2020]. The test sets contains 43 queries and 8.8 million passages. The goal is to rank the passages according to their relevance to each query.

Table 1: Kendall’s Tau Scores for GPT-4o and GPT-4o mini

Model	Method	Kendall’s Tau
GPT-4o	pointwise	0.9249
	external pointwise	0.9461
	external bubble m=8	0.9139
	external merge m=8	0.8798
	external bubble m=16	0.7808
	external merge m=16	0.5914
	quicksort vote=1	0.8347
	quicksort vote=3	0.8391
GPT-4o mini	pointwise	0.7159
	external pointwise	0.6156
	external bubble m=4	0.7124
	external merge m=4	0.6281
	external bubble m=8	0.6501
	external merge m=8	0.1721
	quicksort vote=1	0.5990
	quicksort vote=3	0.5182

4.2 Implementation and Metrics

We implemented all sorting algorithms in Python and invoked the OpenAI endpoint through the structured API. The input JSON schema was designed following the best practices of chain-of-thought prompting Wei et al. [2022], as recommended in the OpenAI structured outputs guide OpenAI [2024]. Our value-based prompt follow the design of passage relevance generation practice Liang et al. [2022] and our comparison-based prompts closely follow the design in Qin et al. [2023], Ma et al. [2023]. For finding the batch size in external pointwise approach, we set the agreement threshold to 60%.

In evaluating sortedness on the NBA dataset, we adopt the well-established Kendall’s Tau metric Kendall [1938], implemented via the `scipy.stats.kendalltau` function. Kendall’s Tau quantifies the correspondence between two rankings, with values near 1 indicating strong agreement and values near -1 indicating strong disagreement. In evaluating the sortedness on DL19, we adopt the standard nDCG@10 as evaluation metrics Wang et al. [2013]. Since ranking 8.8 million passages per query with an LLM is infeasible, we instead re-rank the top 100 passages retrieved by BM25 Robertson et al. [2009]. This is a standard practice, and we directly use the `pyserini` toolkit Lin et al. [2021] for BM25 retrieval. The normalized discounted cumulative gain (nDCG) yields values from 0.0 to 1.0, where higher values indicate rankings that are closer to the ideal order.

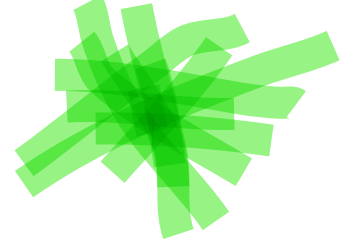
4.3 Experimental Results

Ranking consistency (NBA heights).

Table 1 reports Kendall’s tau score comparing predicted orderings with ground truth. Overall, GPT-4o achieves higher rank consistency than GPT-4o mini across all methods. For GPT-4o, the best-performing setting is external pointwise, which automatically find the batch size with our proposed Algorithm 1, followed closely by pointwise. This suggests that the larger model, when applied to factual data contained in Wikipedia (which is highly likely part of its training corpus), benefits from directly inferring values—whether through reasoning or memorization—as well as from modest externalization that provides related information. We can also observe that as the batch size increase in external comparison-based approach, the quality of the ranking drops. Our proposed quick sort

Table 2: nDCG@10 Scores for GPT-4o and GPT-4o mini

Model	Method	nDCG@10
GPT-4o	pointwise	0.6851
	external pointwise	0.6710
	external bubble $m=4$	0.7313
	external merge $m=4$	0.7396
	quicksort vote=1	0.7352
	quicksort vote=3	0.7408
GPT-4o-mini	pointwise	0.7028
	external pointwise	0.6520
	external bubble $m=4$	0.7153
	external merge $m=4$	0.7141
	quicksort vote=1	0.7095
	quicksort vote=3	0.6936



with majority voting variant achieves more stable and accurate rankings than quick sort, indicating that controlled pairwise aggregation can effectively improve consistency.

For GPT-4o mini, *pointwise* is strongest, followed by *external bubble $m=4$* . In fact, *external pointwise* also operates with a batch size of four, as determined by Algorithm 1. This indicates that the smaller language model has more limited capacity, benefiting primarily from focusing on a single entity. The excessive compute used in *external bubble $m=4$* helps stabilize comparisons and slightly boosts performance. Similarly to GPT-4o, increasing the batch size further leads to a decrease in ranking quality. Moreover, we see a sharp drop of quality with external merge sort $m=9$, suggesting that the smaller model is more sensitive to aggregation noise introduced by larger external comparisons.

Passage retrieval quality (nDCG@10). Table 2 shows nDCG@10. Compared to models, GPT-4o consistently matches or exceeds GPT-4o mini in the ranking of passages. For GPT-4o, the best approach is our proposed quick sort with majority votes, indicating that limited pairwise aggregation improves passage placements; Our proposed *external merge sort* is a close second (0.7396). The pointwise and external pointwise approaches yield the lowest scores, which is consistent with observations reported in prior works on passage ranking. When the task is not centered on deriving factual information, value-based approaches are less effective, as they lack the comparative reasoning needed to capture subtle relevance signals.

For GPT4o-mini, the strongest result is *external bubble sort*, followed closely by our proposed *external merge sort*. However, as we later demonstrate, our *external merge sort* is significantly more efficient, making it a more practical choice despite the small accuracy gap.

Trade-off between Sorting Accuracy and Computation

Figure 1 illustrates the relationship between the sorting accuracy (Kendall’s tau or nDCG@10) and computational cost (measured in terms of the number of tokens consumed) across four experimental settings (NBA and DL19 for GPT-4o and GPT-4o mini). In many cases, the curves follow a clear log-linear trend: accuracy improves rapidly with small increases in tokens but quickly plateaus as token usage grows. For the NBA dataset, the fitted curve excludes the pointwise and external pointwise approaches, as we suspect their strong performance stems from memorization of factual data (e.g., Wikipedia), rather than from generalizable reasoning ability. For DL19, the fitted curve include all points. We also highlight that our proposed external merge sort approach, when used with a moderate batch size, consistently achieves strong performance and efficiency across different settings.

Summary Across both datasets, we observe a clear trade-off between sorting accuracy and computation. For factual tasks such as NBA player heights, GPT-4o achieves very high Kendall’s tau using external pointwise approach with minimal computation; GPT-4o mini with limited capability benefit from focusing on single entity and hence pointwise approach achieve the best score. Although the high scores achieved by value-based approaches likely reflect memorization, we argue that for tasks involving factual data derivation, value-based methods remain the most effective choice. For DL19 passage ranking, where queries require semantic matching beyond factual recall, both models follow the expected log-linear curve. Here, pairwise aggregation methods such as our proposed quicksort with majority voting, along with external merge sort, provide the strongest balance of accuracy and

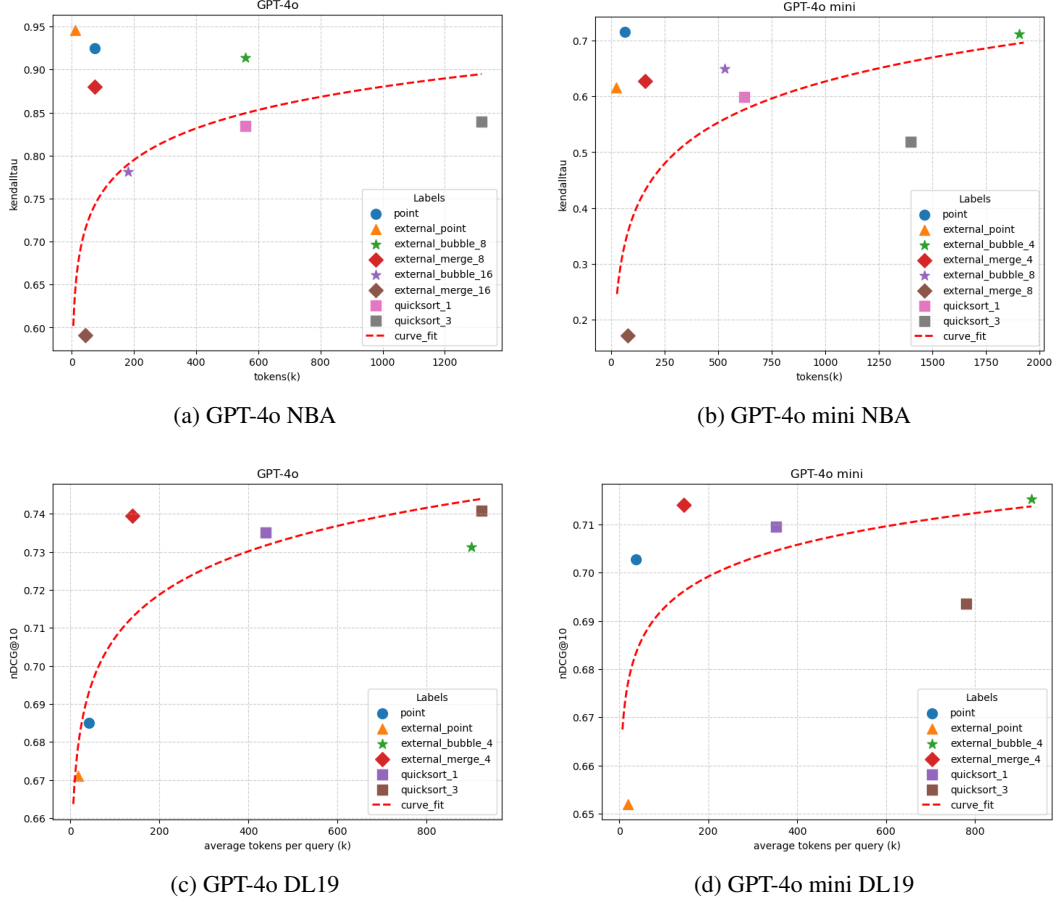


Figure 1: Sorting accuracy v.s. the number of tokens across four experimental settings. These figures display a log-linear growth pattern, with accuracy increasing quickly at first before plateauing.

efficiency. Overall, these results suggest that while larger models are more robust and can tolerate broader aggregation, smaller models require careful tuning of batch size to avoid noise.

5 Discussions

In this work, we presented the LLM ORDER BY operator as a logical abstraction and systematically studied its physical implementations. Through a unified experimental framework across diverse sorting tasks, we demonstrated that no single access path—pointwise, pairwise, or listwise—universally dominates (also referred to as value-based or comparison-based). Instead, the effectiveness of an implementation depends strongly on the query characteristics and underlying data, highlighting the importance of adaptive access-path selection in practice.

We introduced several new algorithmic designs, including a batch-size determination strategy for value-based methods, a majority-voting mechanism for comparison-based approach, and a two-way external merge sort tailored for LLMs. Among these, the external merge sort method consistently achieved high accuracy–efficiency trade-offs for all datasets and the majority voting mechanism consistently improved the robustness of pairwise comparisons for GPT-4o. Beyond algorithmic design, our analysis suggests a log-linear scaling relationship between compute cost and ordering quality, offering a first step toward principled cost models for LLM-based query processing.

References

- Ashwin Alaparthi, Paul Loh, and Ryan Marcus. Scalellm: A technique for scalable llm-augmented data systems. In *Companion of the 2025 International Conference on Management of Data*, pages 11–14, 2025.
- Amazon Web Services. Bringing generative ai to the data warehouse with amazon bedrock and amazon redshift, February 2024. URL <https://repost.aws/articles/ARJsZlMEpRti6xoM-OfsBmw/bringing-generative-ai-to-the-data-warehouse-with-amazon-bedrock-and-amazon-redshift>. AWS re:Post article; accessed: 2025-08-17.
- Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M. Voorhees. Overview of the TREC 2019 deep learning track. *CoRR*, abs/2003.07820, 2020. URL <https://arxiv.org/abs/2003.07820>.
- Andrew Drozdov, Honglei Zhuang, Zhuyun Dai, Zhen Qin, Razieh Rahimi, Xuanhui Wang, Dana Alon, Mohit Iyyer, Andrew McCallum, Donald Metzler, et al. Parade: Passage ranking using demonstrations with large language models. *arXiv preprint arXiv:2310.14408*, 2023.
- Avrilia Floratou, Fotis Psallidas, Fuheng Zhao, Shaleen Deep, Gunther Hagleither, Wangda Tan, Joyce Cahoon, Rana Alotaibi, Jordan Henkel, Abhik Singla, et al. Nl2sql is a solved problem... not! In *CIDR*, 2024.
- Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.
- Parker Glenn, Parag Pravin Dakle, Liang Wang, and Preethi Raghavan. Blendsql: A scalable dialect for unifying hybrid question answering in relational algebra. *arXiv preprint arXiv:2402.17882*, 2024.
- Yue Gong, Chuan Lei, Xiao Qin, Kapil Vaidya, Balakrishnan Narayanaswamy, and Tim Kraska. Sqlens: An end-to-end framework for error detection and correction in text-to-sql. *arXiv preprint arXiv:2506.04494*, 2025.
- Google Cloud. Introduction to ai and ml in bigquery, August 2025. URL <https://cloud.google.com/bigquery/docs/bqml-introduction>. Accessed: 2025-08-17.
- Zijian He, Reyna Abhyankar, Vikranth Srivatsa, and Yiyang Zhang. Cognify: Supercharging gen-ai workflows with hierarchical autotuning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 932–943, 2025.
- Maurice G Kendall. A new measure of rank correlation. *Biometrika*, 30(1-2):81–93, 1938.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36:42330–42357, 2023.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.
- Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2021.
- Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. Palimpzest: Optimizing ai-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*, page 2, 2025.

- Jian Luo, Xuanang Chen, Ben He, and Le Sun. Prp-graph: Pairwise ranking prompting to llms with graph aggregation for effective text re-ranking. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5766–5776, 2024.
- Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. Zero-shot listwise document reranking with a large language model. *arXiv preprint arXiv:2305.02156*, 2023.
- OpenAI. Structured model outputs. OpenAI API Guide, 2024. URL <https://platform.openai.com/docs/guides/structured-outputs>. Structured outputs ensure model responses adhere to a supplied JSON Schema. Accessed: 2025-08-25.
- OpenIntro. Nba player heights (2008–09 season). R package openintro, dataset nba_heights, 2025. Available at https://www.openintro.org/data/index.php?data=nba_heights, accessed 2025-08-25.
- Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. Lotus: Enabling semantic queries with llms over tables of unstructured and structured data. *arXiv e-prints*, pages arXiv–2407, 2024.
- Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. Rankvicuna: Zero-shot listwise document reranking with open-source large language models. *arXiv preprint arXiv:2309.15088*, 2023.
- Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, et al. Large language models are effective text rankers with pairwise ranking prompting. *arXiv preprint arXiv:2306.17563*, 2023.
- Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.
- Devendra Singh Sachan, Mike Lewis, Mandar Joshi, Armen Aghajanyan, Wen-tau Yih, Joelle Pineau, and Luke Zettlemoyer. Improving passage retrieval with zero-shot question generation. *arXiv preprint arXiv:2204.07496*, 2022.
- Dario Satriani, Enzo Veltri, Donatello Santoro, Sara Rosato, Simone Varriale, and Paolo Papotti. Logical and physical optimizations for sql query execution over large language models. *Proceedings of the ACM on Management of Data*, 3(3):1–28, 2025.
- P Griffiths Selinger, Morton M Astrahan, Donald D Chamberlin, Raymond A Lorie, and Thomas G Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*, pages 23–34, 1979.
- Nihar B Shah and Martin J Wainwright. Simple, robust and optimal ranking from pairwise comparisons. *Journal of machine learning research*, 18(199):1–38, 2018.
- Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. Doctel: Agentic query rewriting and evaluation for complex document processing. *arXiv preprint arXiv:2410.12189*, 2024.
- Snowflake, Inc. Snowflake cortex aisql (including llm functions), June 2025. URL <https://docs.snowflake.com/user-guide/snowflake-cortex/aisql?lang=de/>. Preview feature documentation; accessed: 2025-08-17.
- Ji Sun, Guoliang Li, Peiyao Zhou, Yihui Ma, Jingzhe Xu, and Yuan Li. Agenticdata: An agentic data analytics system for heterogeneous data. *arXiv preprint arXiv:2508.05002*, 2025.
- Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is chatgpt good at search? investigating large language models as re-ranking agents. *arXiv preprint arXiv:2304.09542*, 2023.
- Yining Wang, Liwei Wang, Yuanzhi Li, Di He, and Tie-Yan Liu. A theoretical analysis of ndcg type ranking measures. In *Conference on learning theory*, pages 25–54. PMLR, 2013.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

- Sirui Zeng and Xifeng Yan. Adl: A declarative language for agent-based chatbots. *arXiv preprint arXiv:2504.14787*, 2025.
- Fuheng Zhao, Jiayue Chen, Lawrence Lim, Ishtiyaque Ahmad, Divyakant Agrawal, and Amr El Abbadi. Llm-sql-solver: Can llms determine sql equivalence? *arXiv preprint arXiv:2312.10321*, 2023.
- Fuheng Zhao, Divyakant Agrawal, and Amr El Abbadi. Hybrid querying over relational databases and large language models. *arXiv preprint arXiv:2408.00884*, 2024a.
- Fuheng Zhao, Shaleen Deep, Fotis Psallidas, Avrielia Floratou, Divyakant Agrawal, and Amr El Abbadi. Sphinteract: Resolving ambiguities in nl2sql through user interaction. *Proceedings of the VLDB Endowment*, 18(4):1145–1158, 2024b.
- Lixi Zhou, Qi Lin, Kanchan Chowdhury, Saif Masood, Alexandre Eichenberger, Hong Min, Alexander Sim, Jie Wang, Yida Wang, Kesheng Wu, et al. Serving deep learning models from relational databases. *Advances in Database Technology-EDBT*, 27(3):717–724, 2024.
- Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. Large language models for information retrieval: A survey. *arXiv preprint arXiv:2308.07107*, 2023.
- Shengyao Zhuang, Honglei Zhuang, Bevan Koopman, and Guido Zuccon. A setwise approach for effective and highly efficient zero-shot ranking with large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 38–47, 2024.
- Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.